МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОТЧЁТ

по лабораторной работе № 2

«Эмпирический анализ временной сложности алгоритмов сортировки. Часть 1: квадратичные алгоритмы (сортировка обменом и сортировка вставками)»

Дисциплина: **Теория алгоритмов**

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 13

Выполнила: студентка группы БИ 2-1

Кугушева Арина Вячеславовна

Проверил: _____

2026

1. Цель работы и задание варианта

Цель работы: освоить методику экспериментального исследования временной сложности алгоритмов и сопоставить полученные эмпирические зависимости с теоретическими асимптотическими оценками на примере простых алгоритмов сортировки.

Задание (вариант 13). Сравнить время работы сортировки обменом («пузырьком», с флагом досрочного выхода) и сортировки вставками на **упорядоченных по возрастанию** целочисленных массивах (тип S) со значениями из диапазона [0; 1000] размером n = 1000, 2000, 3000, 4000, 5000. Каждое измерение повторить k = 5 раз и взять медиану.

№	Размеры массива n	Тип данных	Диапазон	Повторов k	Доп. задание
13	1000, 2000, 3000, 4000, 5000	S (упорядоченные)	[0; 1000]	5	M1

Дополнительное задание M1: дополнительно к времени подсчитать число сравнений и число обменов (сдвигов) для каждого алгоритма и проверить соотношения $\approx n^2/2$ и $\approx n^2/4$.

Замечание. На упорядоченных данных (лучший случай) число сравнений и обменов линейно по n, поэтому соотношения $n^2/2$ и $n^2/4$ на данных варианта выполняться не могут. Для их проверки подсчёт числа операций дополнительно выполнен на случайных массивах тех же размеров, а время на случайных данных измерено для сравнения с квадратичной оценкой среднего случая.

2. Краткое описание алгоритмов и теоретические оценки

Сортировка пузырьком. Соседние элементы сравниваются и при нарушении порядка меняются местами; после i-го прохода i наибольших элементов стоят на своих местах. Флаг `swapped` позволяет завершить работу, если за проход не было ни одного обмена; на упорядоченном массиве достаточно одного прохода ($n - 1$ сравнений).

Сортировка вставками. Левая часть массива считается отсортированной; очередной элемент (ключ) вставляется на своё место с помощью сдвига больших элементов вправо. Число сдвигов равно числу инверсий массива, поэтому на упорядоченных данных сдвигов нет, а сравнений $n - 1$.

Таблица 1. Теоретические оценки

Алгоритм	Лучший случай	Средний случай	Худший случай	Доп. память	Устойчивость
Пузырьком (с флагом)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Вставками	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да

Упорядоченный массив — лучший случай для обоих алгоритмов, поэтому ожидается линейный рост: $T(2n)/T(n) \approx 2$. Для случайных данных (средний случай) ожидается $T(2n)/T(n) \approx 4$. В среднем пузырёк выполняет $\approx n^2/2$ сравнений и $\approx n^2/4$ обменов, вставки — $\approx n^2/4$ сравнений и $\approx n^2/4$ сдвигов (число инверсий случайной перестановки равно $n(n-1)/4$).

Методика измерений. Время измеряется `time.perf_counter()`; данные одинаковы для всех алгоритмов (фиксированный seed), замер повторяется k = 5 раз, берётся медиана; результат проверяется по `sorted()` вне замера. Счётчики сравнений и обменов вынесены в отдельные версии функций, чтобы не искажать время.

3. Блок-схемы алгоритмов

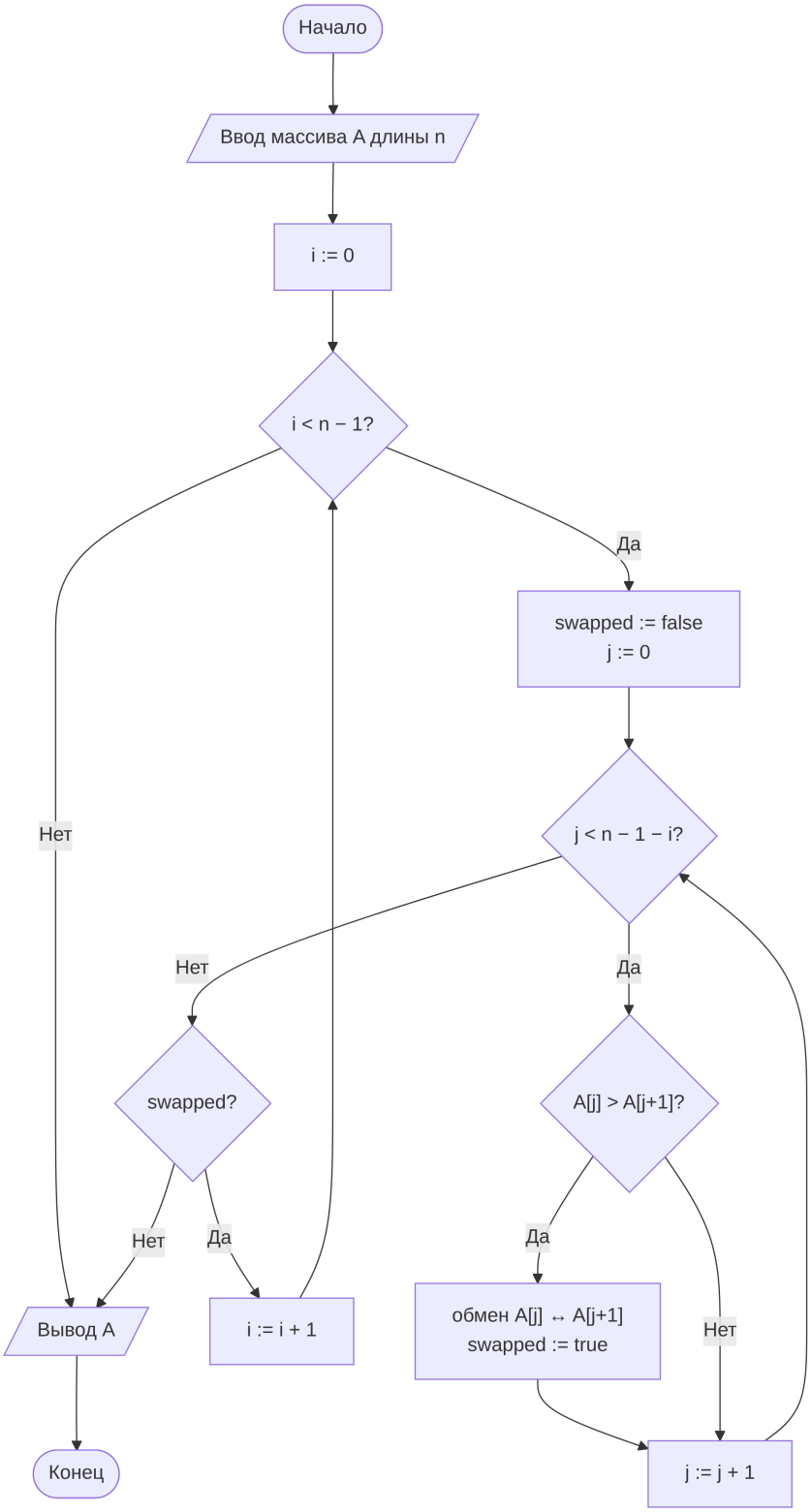


Рис. 1. Блок-схема сортировки пузырьком

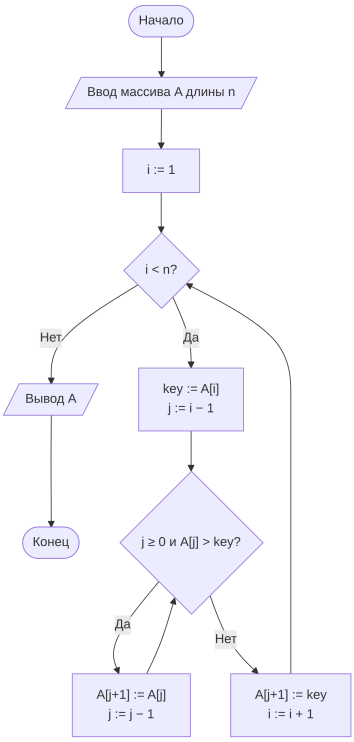


Рис. 2. Блок-схема сортировки вставками

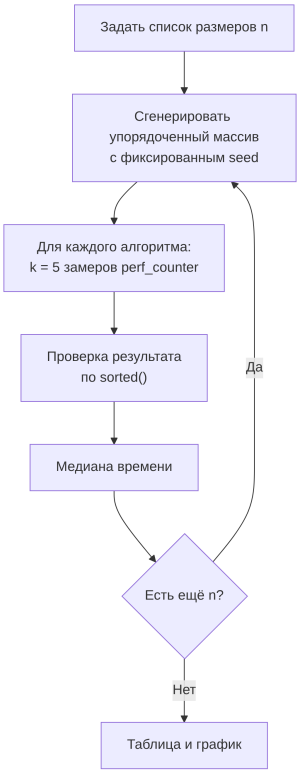


Рис. 3. Блок-схема экспериментального стенда

4. Листинг программы

Файл `lr2_variant13.py` (генерация данных, алгоритмы, замер времени и построение графика собраны в одном модуле).

```
"""
Лабораторная работа № 2. Вариант 13.
Дисциплина: Теория алгоритмов.
Выполнила: Кугушева Арина Вячеславовна, группа БИ 2-1.

Эмпирический анализ временной сложности квадратичных сортировок:
сортировка обменом («пузырьком», с флагом) и сортировка вставками.

Исходные данные варианта 13:
    размеры n = 1000, 2000, 3000, 4000, 5000;
    тип данных S (упорядоченные), диапазон значений [0; 1000];
    число повторов k = 5 (в качестве результата берётся медиана);
    дополнительное задание M1 - подсчёт числа сравнений и числа
    обменов (сдвигов) для каждого алгоритма, проверка соотношений
    ~ n^2/2 и ~ n^2/4.

Запуск:  python lr2_variant13.py          (график сохраняется в lr2_plot.png)
         python lr2_variant13.py --show   (дополнительно показать график)
Требуется: matplotlib.
"""
import platform
import random
import statistics
import sys
import time

import matplotlib

if "--show" not in sys.argv:
    matplotlib.use("Agg")          # без окна: график только сохраняется в файл
import matplotlib.pyplot as plt

# ----------------------------- параметры варианта -----------------------------
SIZES = [1000, 2000, 3000, 4000, 5000]
KIND = "sorted"                    # тип S - упорядоченные данные
LO, HI = 0, 1000                   # диапазон значений
REPEATS = 5                        # число повторов k
SEED = 42


# ------------------------------ генерация данных ------------------------------
def generate_data(n, kind="random", lo=LO, hi=HI, seed=SEED):
    """Генерирует массив длины n заданного типа.

    kind: "random" (R), "sorted" (S), "reversed" (V).
    Генератор с фиксированным seed делает эксперимент воспроизводимым.
    """
    rng = random.Random(seed + n)
    data = [rng.randint(lo, hi) for _ in range(n)]
    if kind == "sorted":
        data.sort()
    elif kind == "reversed":
        data.sort(reverse=True)
    return data


# ------------------------------ алгоритмы сортировки --------------------------
# Все функции НЕ изменяют исходный массив и возвращают новый список.
def bubble_sort(arr):
    """Сортировка пузырьком с флагом досрочного выхода."""
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a


def insertion_sort(arr):
    """Сортировка вставками."""
    a = arr.copy()
    for i in range(1, len(a)):
        key = a[i]
```

```
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a


# Версии со счётчиками для задания M1 (отдельные, чтобы счётчики не влияли
# на измерение времени). Возвращают (отсортированный список, сравнений, обменов).
def bubble_sort_counted(arr):
    """Пузырёк с флагом: считает сравнения и обмены."""
    a = arr.copy()
    n = len(a)
    comparisons = swaps = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return a, comparisons, swaps


def insertion_sort_counted(arr):
    """Вставки: считает сравнения элементов и сдвиги."""
    a = arr.copy()
    comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1              # сравнение a[j] > key
            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1
                j -= 1
            else:
                break
        a[j + 1] = key
    return a, comparisons, shifts


# ------------------------------ замер времени --------------------------------
def measure(sort_func, data, repeats=REPEATS):
    """Медианное время работы sort_func на данных data, с (perf_counter)."""
    expected = sorted(data)
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = sort_func(data)
        times.append(time.perf_counter() - start)
        assert result == expected, f"{sort_func.__name__}: ошибка сортировки"
    return statistics.median(times)


def run_series(kind):
    """Времена обоих алгоритмов для всех n из SIZES на данных типа kind."""
    res = {"Пузырьком": [], "Вставками": []}
    for n in SIZES:
        data = generate_data(n, kind)
        res["Пузырьком"].append(measure(bubble_sort, data))
        res["Вставками"].append(measure(insertion_sort, data))
    return res


def system_info():
    """Характеристики вычислительной системы (для отчёта)."""
    cpu = platform.processor() or platform.machine()
    try:                                          # Linux: точное имя процессора
        with open("/proc/cpuinfo", encoding="utf-8") as f:
            for line in f:
                if line.startswith("model name"):
                    cpu = line.split(":", 1)[1].strip()
                    break
    except OSError:
        pass
    return (f"ОС: {platform.system()} {platform.release()}; процессор: {cpu}; "
            f"Python {platform.python_version()}")
def ratio_line(times, label):
    """Отношения T(2n)/T(n) для пар 1000->2000 и 2000->4000."""
```

```
    t = dict(zip(SIZES, times))
    parts = []
    for n in (1000, 2000):
        parts.append(f"{n}->{2 * n}: {t[2 * n] / t[n]:.2f}")
    return f"{label:<11}" + "   ".join(parts)


# ---------------------------------- main ----------------------------------
def main():
    print("Лабораторная работа № 2, вариант 13")
    print(system_info())
    print(f"n = {SIZES}, тип данных: {KIND} (упорядоченные), диапазон [{LO}; {HI}], "
          f"k = {REPEATS}\n")

    # 1. Основной эксперимент варианта: упорядоченные данные (S)
    res_s = run_series("sorted")
    print("Таблица 1. Медианное время сортировки УПОРЯДОЧЕННЫХ массивов, мс")
    print(f"{'n':>6}{'Пузырьком':>13}{'Вставками':>13}{'Пузырёк/Вставки':>18}"
          f"{'T/n, мкс (п.)':>16}{'T/n, мкс (в.)':>16}")
    for i, n in enumerate(SIZES):
        tb, ti = res_s["Пузырьком"][i], res_s["Вставками"][i]
        print(f"{n:>6}{tb * 1e3:>13.4f}{ti * 1e3:>13.4f}{tb / ti:>18.2f}"
              f"{tb / n * 1e6:>16.4f}{ti / n * 1e6:>16.4f}")
    print("\nОтношения T(2n)/T(n) на упорядоченных данных (теория для Θ(n): 2)")
    for name, times in res_s.items():
        print(ratio_line(times, name))

    # 2. M1: сравнения и обмены на упорядоченных данных (вариант 13)
    print("\nТаблица 2 (M1). Сравнения и обмены на УПОРЯДОЧЕННЫХ данных")
    print(f"{'n':>6}{'Пузырёк: сравн.':>18}{'обмены':>9}{'Вставки: сравн.':>18}{'сдвиги':>9}")
    for n in SIZES:
        data = generate_data(n, "sorted")
        _, cb, sb = bubble_sort_counted(data)
        _, ci, si = insertion_sort_counted(data)
        print(f"{n:>6}{cb:>18}{sb:>9}{ci:>18}{si:>9}")

    # 3. M1: проверка соотношений ~ n^2/2 и ~ n^2/4 на СЛУЧАЙНЫХ данных
    print("\nТаблица 3 (M1). Сравнения и обмены на СЛУЧАЙНЫХ данных, "
          "отношение к n^2")
    print(f"{'n':>6}{'П.сравн./n²':>13}{'П.обмены/n²':>13}"
          f"{'В.сравн./n²':>13}{'В.сдвиги/n²':>13}")
    for n in SIZES:
        data = generate_data(n, "random")
        _, cb, sb = bubble_sort_counted(data)
        _, ci, si = insertion_sort_counted(data)
        print(f"{n:>6}{cb / n**2:>13.4f}{sb / n**2:>13.4f}"
              f"{ci / n**2:>13.4f}{si / n**2:>13.4f}")
    print("Теория: 0.5000 / 0.2500 / 0.2500 / 0.2500")

    # 4. Дополнение: время на случайных данных (проверка квадратичного роста)
    res_r = run_series("random")
    print("\nТаблица 4. Медианное время сортировки СЛУЧАЙНЫХ массивов, с")
    print(f"{'n':>6}{'Пузырьком':>13}{'Вставками':>13}{'Пузырёк/Вставки':>18}"
          f"{'T/n² п., нс':>14}{'T/n² в., нс':>14}")
    for i, n in enumerate(SIZES):
        tb, ti = res_r["Пузырьком"][i], res_r["Вставками"][i]
        print(f"{n:>6}{tb:>13.4f}{ti:>13.4f}{tb / ti:>18.2f}"
              f"{tb / n**2 * 1e9:>14.2f}{ti / n**2 * 1e9:>14.2f}")
    print("\nОтношения T(2n)/T(n) на случайных данных (теория для Θ(n²): 4)")
    for name, times in res_r.items():
        print(ratio_line(times, name))

    # 5. График T(n)
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.3))
    for ax, res, title in [(axes[0], res_s, "Упорядоченные данные (вариант 13)"),
                           (axes[1], res_r, "Случайные данные (для сравнения)")]:
        for name, times in res.items():
            ax.plot(SIZES, times, marker="o", label=name)
        ax.set_xlabel("Размер массива n")
        ax.set_ylabel("Время, с")
        ax.set_title(title)
        ax.grid(True)
        ax.legend()
    fig.tight_layout()
    fig.savefig("lr2_plot.png", dpi=150)
    print("\nГрафик сохранён в файл lr2_plot.png")
    if "--show" in sys.argv:
        plt.show()


if __name__ == "__main__":
    main()
```

5. Характеристики вычислительной системы

Параметр	Значение
Процессор	Intel(R) Xeon(R) Processor @ 2.30GHz, 2 логических ядра
Оперативная память	8 ГБ
Операционная система	Linux (Ubuntu 24.04)
Интерпретатор	Python 3.13.18
Таймер	time.perf_counter()

Абсолютные значения времени зависят от системы; на другом компьютере они будут отличаться, но соотношения между ними (отношения T(2n)/T(n), сравнение алгоритмов) должны сохраниться. Программа при запуске выводит характеристики системы автоматически.

6. Результаты измерений

Таблица 2. Медианное время сортировки упорядоченных массивов (тип S), мс

n	Пузырьком	Вставками	Отношение (пузырёк / вставки)
1000	0,0298	0,0536	0,56
2000	0,0580	0,1128	0,51
3000	0,0888	0,1715	0,52
4000	0,1157	0,2391	0,48
5000	0,1647	0,2841	0,58

Таблица 3 (M1). Число сравнений и обменов (сдвигов) на упорядоченных данных

n	Пузырёк: сравнения	Пузырёк: обмены	Вставки: сравнения	Вставки: сдвиги
1000	999	0	999	0
2000	1999	0	1999	0
3000	2999	0	2999	0
4000	3999	0	3999	0
5000	4999	0	4999	0

Таблица 4 (M1). Проверка соотношений на случайных данных: число операций / n^2

n	Пузырёк: сравнения	Пузырёк: обмены	Вставки: сравнения	Вставки: сдвиги
1000	0,4994	0,2512	0,2522	0,2512
2000	0,4996	0,2532	0,2537	0,2532
3000	0,4997	0,2505	0,2508	0,2505
4000	0,4998	0,2451	0,2454	0,2451
5000	0,4999	0,2536	0,2538	0,2536
Теория	0,5	0,25	0,25	0,25

Таблица 5. Для сравнения: медианное время сортировки случайных массивов, с

n	Пузырьком	Вставками	Отношение (пузырёк / вставки)
1000	0,0270	0,0122	2,22
2000	0,1033	0,0509	2,03
3000	0,2334	0,1123	2,08
4000	0,4705	0,2149	2,19
5000	0,7119	0,3212	2,22

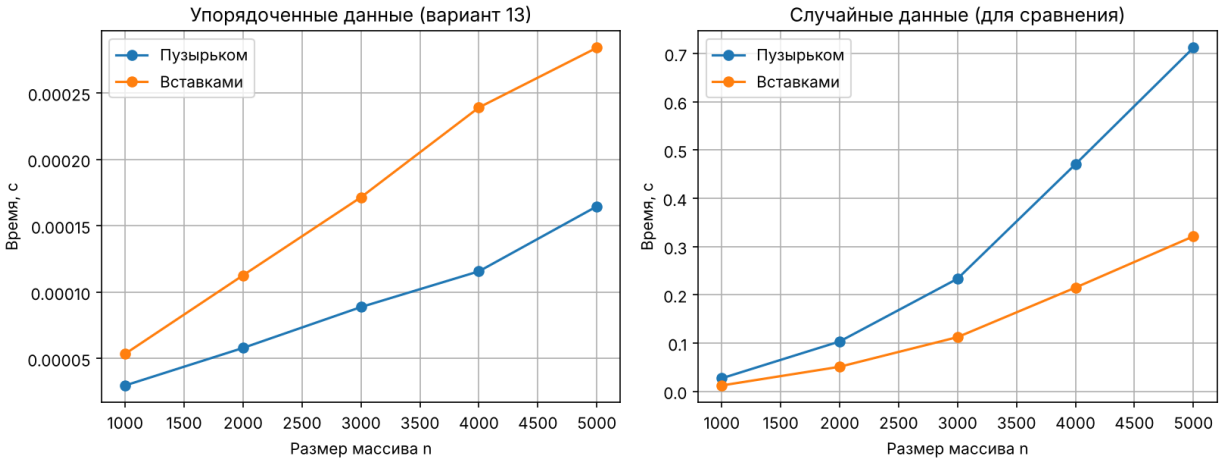


Рис. 4. Зависимость T(n): слева — упорядоченные данные (вариант 13), справа — случайные данные

7. Расчёт отношений T(2n)/T(n), T(n)/n и T(n)/n²

Таблица 6. Отношение T(2n)/T(n)

Переход	Данные	Пузырьком	Вставками	Теория
1000 → 2000	упорядоченные (S)	1,95	2,10	2 ($\Theta(n)$)
2000 → 4000	упорядоченные (S)	1,99	2,12	2 ($\Theta(n)$)
1000 → 2000	случайные (R)	3,82	4,19	4 ($\Theta(n^2)$)
2000 → 4000	случайные (R)	4,56	4,22	4 ($\Theta(n^2)$)

Таблица 7. Проверка постоянства T(n)/n (упорядоченные данные), мкс

n	Пузырьком	Вставками
1000	0,0298	0,0536
2000	0,0290	0,0564
3000	0,0296	0,0572
4000	0,0289	0,0598
5000	0,0329	0,0568

Таблица 8. Проверка постоянства $T(n)/n^2$ (случайные данные), нс

n	Пузырьком	Вставками
1000	27,01	12,15
2000	25,82	12,73
3000	25,93	12,48
4000	29,41	13,43
5000	28,48	12,85

8. Выводы

1. **Упорядоченные данные — лучший случай.** Оба алгоритма выполняют ровно $n - 1$ сравнений и ни одного обмена (сдвига) (табл. 3), то есть работают за $\Theta(n)$. Это подтверждает время: при удвоении n оно возрастает в 1,95–2,12 раза при теоретическом значении 2 (а не 4, как для квадратичного роста), а $T(n)/n$ практически постоянно (0,0289–0,0329 мкс у пузырька, 0,0536–0,0598 мкс у вставок). Небольшие отклонения объясняются тем, что время порядка 0,03–0,3 мс сравнимо с погрешностью таймера и накладными расходами интерпретатора.
2. **На упорядоченных данных пузырёк быстрее вставок** примерно в 1,7–2 раза (отношение 0,48–0,58), хотя число сравнений одинаково. Причина — в константе: на каждом шаге внешнего цикла сортировка вставками выполняет дополнительные присваивания (`key = a[i]`, `j = i - 1`, `a[j + 1] = key`) и запускает цикл `while`, тогда как внутренний цикл пузырька только сравнивает соседние элементы. Теоретическая асимптотика этого различия не отражает.
3. **Соотношения М1.** На случайных данных пузырёк выполняет $\approx 0{,}5n^2$ сравнений ($0{,}4994–0{,}4999 \cdot n^2$) и $\approx 0{,}25n^2$ обменов ($0{,}245–0{,}254 \cdot n^2$), вставки — $\approx 0{,}25n^2$ сравнений и $\approx 0{,}25n^2$ сдвигов (табл. 4). Число обменов пузырька и сдвигов вставок совпадает: оно равно числу инверсий $n(n - 1)/4$. Вставки делают вдвое меньше сравнений, чем пузырёк.
4. **Квадратичный рост на случайных данных.** $T(n)/n^2$ почти постоянно (25,82–29,41 нс у пузырька, 12,15–13,43 нс у вставок), а при удвоении n время возрастает в 3,82–4,56 раза (пузырёк) и 4,19–4,22 раза (вставки) при теоретическом значении 4. Это подтверждает средний случай $\Theta(n^2)$. На случайных данных вставки быстрее пузырька в 2,0–2,2 раза: вдвое меньше сравнений и более дешёвый сдвиг по сравнению с обменом.
5. **Практическое применение.** Алгоритмы пригодны для небольших массивов и почти упорядоченных данных (например, дописывание нескольких записей в уже отсортированный журнал): 5000 упорядоченных элементов обрабатываются за доли миллисекунды. На случайных данных 5000 элементов требуют 0,3–0,7 с, а экстраполяция на 100 000 элементов (рост в 400 раз) даёт $\approx$ 2 минуты для вставок и $\approx$ 5 минут для пузырька, что неприемлемо; для больших массивов нужны алгоритмы со сложностью $O(n \log n)$.

9. Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Гл. 2, 3.
2. Кнут Д. Э. Искусство программирования. Т. 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2017.
3. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013. — Гл. 2.

4. Скиена С. Алгоритмы. Руководство по разработке. — 3-е изд. — СПб.: БХВ-Петербург, 2022.
5. Документация Python: модуль time [Электронный ресурс]. — URL: https://docs.python.org/3/library/time.html